ECE 157C - Homework #1

Build a CSV Question-Answering AI Agent with LangGraph

Magnolia Saur | April 17, 2026

1. Setup and Execution

1.1 Setup Instructions

The following Python packages are required and listed in requirements.txt:

- langgraph>=0.2.60
- langchain-openai>=0.2.14
- langchain-core>=0.3.20
- openai>=1.55.0
- pandas>=2.2.3
- python-dotenv>=1.0.1

To install all dependencies, run:

```
pip install -r requirements.txt
```

1.2 How to Run the Program

From the project directory, run:

```
python test_agent.py
```

When prompted, enter one of the following:

- **housing data** - runs 9 questions against housing.csv and saves results to housing_results.csv
- **plant data** - runs 9 questions against plant_growth_data.csv and saves results to plant_results.csv

Any other input will print an error and exit. Results are appended to the output CSV on each run, with a trial separator row automatically inserted.

1.3 API Configuration

This project uses the OpenAI API. To configure access:

- Create a file named .env in the project root folder
- Add you key: `OPENAI_API_KEY=your_key_here`
- Ensure your account has active billing and quota enables

The agent loads the API key automatically from the .env file at runtime. If the key is missing or invalid, the agent returns a structured FAIL response instead of crashing.

2. Prompt Log

The following prompts were used with CodeX (VS Code) to build, debug, and improve the agent during development.

Prompt 1 - Initial Agent Scaffolding

Goal: Generate the full project file structure for the LangGraph Agent

Prompt Given: `I want to build an AI Agent using LangGraph. The agent should take a natural-language question, read a CSV dataset, generate Python code, execute that code, evaluate whether the result is correct, generate a natural-langauge answer, and return the final output. If the evaluation passes, it should produce the final answer. If it fails, it should retry the code generation step. I plan to use the OpenAI GPT-5.4 mini model, and the agent should read the API key from a .env file. Please create the following files: agent.py, modes.py, test_agent.py, requirements.txt, README.md. In the test)agent.py, I want to run the agent with different pairs of questions and CSV tables using this function signature: def run_agent(question: str, csv_path: str) ->dict: The agent output should follow this format: {}"generated_code": str, "execution_code": object, "evaluation": "PASS" or "FAIL", "final_answer": str}`

Outcome: Helpful. Codex generates all five files with the full LangGraph structure with the retry loop and FINAL_RESULT convention for generated code. The agent structure matched the assignment requirements.

Prompt 2 - Add Housing Dataset Questions

Goal: Update test_agent.py to run all 9 required housing.csv questions

Prompt Given: `In test_agent.py, include test cases using housing.csv with the following questions: 1. What is the average median house value across the dataset? 2. Which ocean proximity category has the highest average median house value? 3. What is the minimum, maximum, and median of median house values? 4. How does median income vary across different ocean proximity categories? 5. Which geographical areas (based on latitude and longitude ranges) have the highest average house prices? 6. How does population density (population per household) relate to median house values? 7. Identity the top 5 most expensive geographical areas and explain the key factors contributing to their high prices? 8. Find coastal areas where house prices are relatively low despite proximity to the ocean. What factors might explain this? 9. Find areas where house prices are significantly higher or lower than expected given median income. Explain possible reasons for these deviations.`

Outcome: Helpful. Codex updated the test_agent.py correctly

Prompt 3 - Fix JSON Serialization Error

Goal: Fix TypeError: Object of type Interval is not JSON serializable on question 5 and 6.

Prompt Given: `In my agent.py or test_agent.py, please update the output formatting so that when results are printed, instead of raw JSON it prints a clean readable summary like:`

```
Question: <question text>
AnswerL <final_answer>
Status: PASS or FAIL
Generated Code: <generated_code>
Do not print execution_result or raw JSON blobs.Just show the question, the final answer, and the status in plain readable text.
```

Outcome: Partially helpful. The fix resolved question 6 on the next run. Question 5 continued to fail in some trials because the generated code still occasionally used pd.cut() with interval_range. Question 5 eventually stabilized and was resolved after multiple runs.

Prompt 4 - Save Terminal Output to CSV

Goal: Save all agent results to a results.CSV file after each run.

Prompt Given: `In my test_agent.py, after running all questions, please add code that saves the results to a CSV file named results.csv in the same directory as the script. The CSV should have these exact columns: question, generated_code, final_answer.`

```
For each question that was run:
   - Question should be a question string
   - Generated_code should be the generated code from the result dict
   - Final_answer should be the final_answer from the result dict
Use pandas to save the CSV. Append results row by row as each question finished, then save once at the end.
```

Outcome: Helpful. Results were saved correctly.

Prompt 5 - Add Trial Separators and Question Numbers to CSV

Goal: Organize CSV output by trial number and question number for easier viewing

Prompt Given: `In my test_agent.py, please update the code that saves to results.csv so that:`

```
   1. Add a question_number column at the beginning that numbers each questions
      (Q1, Q2, Q3, etc)
   2. Between each trial, insert a blank separator row where the
      question_number column says "--- Trial X ---" and all other columns are
      empty. The trial number should auto-increment based on how many
      separators already exist in the file.
   3. Keep all existing columns exactly as they are: question, generated_code,
      final_answer - do not add or remove any other columns.
   4. Always append to the existing results.csv, never overwrite it.

The result should look like:
Question_number, question, generated_code, final_answer
--- Trial 1 ---,,,
Q1, What is the average…, <code>, <answer>
Q2, Which ocean proximity…, <code>, <answer>
--- Trial 2 ---,,,
Q1, What is the average…, <code>, <answer>
```

Outcome: Helpful. This caused a KeyError on the first run after implementation because the existing CSV file was missing the question_number column but was fixed by deleting the pre-existing CSV file.

Prompt 6 - Rename CSV file and Reformat CSV to Match Terminal

Goal: Rename the CSV file to match the dataset and replace the multi-column CSV format with a single output column that mirrors the terminal print format exactly, making the results file easier to read.

Prompt Given: `In my test_agent.py, please rename all references to results.csv to housing_results.csv. This includes the file path where results are read from, written to, and appended to.`

```
In test_agent.py, please update the code that saves to housing_results.csv so
that each row is formatted to match the terminal output exactly. Save each
question as a single formatted text block in one column called "output", like
this:
Question: <question>
Answer: <final_answer>
Generated Code: <generated_code>
So the CSV should have these columns: question_number, output where output
contains the full formatted text block for that question, exactly as it appears
in the terminal.
```

Outcome: Helpful. The reformatted CSV now stores each question as a readable block matching the terminal output exactly, making it much easier to review results across trials without needing to parse multiple columns. Renaming to match the dataset makes it easier to distinguish between questions.

Prompt 7 - Incorporate New Dataset and Allow Dataset Selection Prompt

Goal: Incorporate plant growth CSV file and question set to agent and allow the user to choose between housing and plants dataset when running test_agent.py

Prompt Given: `In test_agent.py, please update the script so that when I run python test_agent.py, it first asks the user to choose which dataset to evaluate:`

```
print("Which dataset would you like to evaluate?")
print("1. housing data")
print("2. plant data")
choice = input("Enter your choice: ").strip().lower()

If the user types "housing data":
- Use housing.csv as the csv_path
- Use these questions:
   "What is the average median house value across the dataset?",
   "Which ocean proximity category has the highest average median house
value?",
   "What is the minimum, maximum, and median of median house values?",
   "How does median income vary across different ocean proximity categories?",
```

```
   "Which geographical areas (based on latitude and longitude ranges) have the
highest average house prices?",
   "How does population density (population per household) relate to median
house values?",
   "Identify the top 5 most expensive geographical areas and explain the key
factors contributing to their high prices.",
   "Find coastal areas where house prices are relatively low despite proximity
to the ocean. What factors might explain this?",
   "Find areas where house prices are significantly higher or lower than
expected given median income. Explain possible reasons for these deviations."
- Save results to housing_results.csv

If the user types "plant data":
- Use plant_growth_data.csv as the csv_path
- Use these questions:
   "What is the average height of plants across the dataset?",
   "Which fertilizer type produces the highest average plant height?",
   "What are the minimum, maximum, and median values of plant height?",
   "How does average plant height vary across different soil types?",
   "Which combination of sunlight exposure and water frequency produces the
highest average growth?",
   "How does temperature relate to plant height across the dataset?",
   "Identify the top 5 growing conditions (based on soil type, fertilizer, and
sunlight) that produce the tallest plants and explain the key factors
contributing to their growth.",
   "Find cases where plants have unusually low growth despite optimal
conditions (high fertilizer, frequent watering, full sunlight). What factors
might explain this?",
   "Identify plants with similar growing conditions but significantly different
heights. What factors might explain these differences?"
- Save results to plant_results.csv

If the user types anything else, print "Invalid choice. Please enter 'housing
data' or 'plant data'." and exit. Keep all existing output formatting and
results.csv logic exactly as it is, just route to the correct dataset,
questions, and output file based on the choice.

After making these changes, please also:
- Update requirements.txt to include any new dependencies that were added
- Update README.md to document the new dataset selection feature, including:
   - how to run the script
   - what to type when prompted
```

```
- what output file each choice produces
- a brief description of both datasets and their questions
```

Outcome: Helpful. Codex implemented the routing cleanly and updated the README to document both dataset options. No new dependencies were required.

Prompt 8 - Update Plant Questions to Match CSV Columns

Goal: Fix plant dataset questions that reference a non-existent Plant_Height column.

Prompt Given: `In test_agent.py, please update the plant data questions list to the following:`

```
"What is the average Sunlight_Hours across the dataset?",
"Which Fertilizer_Type is associated with the highest average Temperature?",
"What are the minimum, maximum, and median values of Humidity?",
"How does average Sunlight_Hours vary across different soil types?",
"Which combination of Water_Frequency and Fertilizer_Type has the highest
average Sunlight_Hours?",
"How does Temperature relate to Humidity across the dataset?",
"Identify the top 5 growing conditions (based on Soil_Type, Fertilizer_Type,
and Water_Frequency) with the highest average Sunlight_Hours and explain the
key environmental factors present in those conditions.",
"Find cases where plants receive high Sunlight_Hours but have low Humidity.
What Soil_Type and Temperature are most common in these cases?",
"Identify plants with similar Soil_Type, Water_Frequency, and Fertilizer_Type
but significantly different Temperature and Humidity values. What patterns
emerge?"
```

Outcome: Helpful. All 9 plant questions passed on the next trial after switching to columns that actually exist in the dataset.

3. Failure Analysis and Debugging

Failure Case #1 - JSON Serialization (Housing Q5)

Question: Which geographical areas (based on latitude and longitude ranges) have the highest average house prices?

Generated code worked on first attempt: No

What went wrong: The generated code used pd.cut() and pd.qcut() to bin latitude and longitude into geographic ranges. These functions return pandas Interval objects, which cannot be serialized to JSON. The agent's internal state passing between nodes uses JSON serialization, causing the error: TypeError: Object of type Interval is not JSON serializable.

How I debugged it: Identified the error in the terminal output and traced it to the pd.cut() call. Recognized that the issue was in the codex prompt rather than the agent infrastructure.

How I improved the prompt: Added explicit rules to the codex prompt in modes.py to not use pd.cut() or pd.qcut(). Instructed instead, use df['lat_group'] = df['latitude'].round(1) for grouping and all values in FINAL_RESULT must be plain strings or numbers.
Whether revised prompt improved the result: Yes. On the next trial, the generated code used .round(1) for geographic grouping and the question passed successfully.

Failure Case #2 - Missing Column in Plant Dataset (Q1-Q4, Q6)

Question: What is the average height of plants across the dataset? / Which fertilizer type produces the highest average plant height? / How does average plant height vary across different soil types? / How does temperature relate to plant height?
Generated code worked on first attempt: No
What went wrong: The initial plant dataset questions referenced plant height as a numeric target variable (e.g., average plant height, minimum/maximum height). The plant_growth_data.csv dataset does not contain a Plant_Height column. The actual columns are Sunlight_Hours, Temperature, and Humidity. The generated code searched for columns containing the word height, found none, and either returned None or fell back to an unrelated column like Growth_Milestone.
How I debugged it: Inspected the FAIL responses in the terminal output and read the evaluator feedback, which stated: the dataset schema has no plant height column. Then inspected the actual CSV to confirm the available columns.
How I improved the prompt: Rewrote all 9 plant questions to use columns that actually exist: Soil_Type, Sunlight_Hours, Water_Frequency, Temperature, and Humidity. Updated the codex prompt in modes.py to specify the exact column names for the plant dataset.
Whether revised prompt improved the result: Yes. All 9 revised plant questions passed on the next trial.

4. Advanced Question Self-Evaluation

The following reflects on the 6 advanced questions across both datasets: 3 from housing and 3 from plants.

Housing Advanced Q7 - Top 5 Most Expensive Geographic Areas

Is the answer correct? Yes. The code grouped coordinates by rounding to 1 decimal place, computed average house value per group, and returned the top 5. The results identify coastal Southern California and Bay Area clusters, which is consistent with known real-world patterns.
Could there be another valid answer? Yes. Grouping by different geographic resolutions (i.e. rounding to 0.5 decimal place) would yield different area definitions and potentially different top 5 results.

Is the question Ambiguous? Yes. The geographic area is not precisely defined.
How to rewrite: Identify the top 5 geographic clusters (defined by rounding latitude and longitude to 1 decimal place) with the highest average median_house_value and explain the key factors contributing to their high prices.

Housing Advanced Q8 - Low-Price Coastal Area

Is the answer correct? Yes. The code correctly identified NEAR OCEAN as the lower-priced coastal category compared to <1H OCEAN, and computed median income, housing age, and population density for the bottom segment of coastal prices
Could there be another valid answer? Yes. A different interpretation might compare all five ocean proximity categories rather than only coastal ones, which could identify INLAND as the lowest-priced category despite it not being coastal at all.
Is the question Ambiguous? Yes. Coastal is not defined. The dataset has NEAR OCEAN, <1H OCEAN, NEAR BAY, and ISLAND as proximity categories that could reasonably be considered coastal.
How to rewrite: Among housing blocks with ocean_proximity of NEAR OCEAN or <1H OCEAN, identify areas in the bottom 25% of median_house_value. What factors might explain this?

Housing Advanced Q9 - Price vs Income Deviation

Is the answer correct? Yes. The code fit a simple linear model ofmedian_house_value from median_income, computed residuals, and identified the most over and underpriced areas.
Could there be another valid answer? Yes. A different expected value model (i.e. multiple regression) would produce different residuals and different area rankings.
Is the question Ambiguous? Yes. Significantly higher or lower is not quantified. The method for computing expected value given income is also not specified.
How to rewrite: Using a simple linear regression of median_house_value on median_income, compute the residual for each block. Identify the top 10 blocks with the largest positive residuals (overpriced) and the top 10 with the largest negative residuals (underpriced). For each group, report the ocean_proximity distribution and average latitude/longitude. Explain what factors might account for the deviations.

Plant Advanced Q7 - Top 5 Growing Conditions by Sunlight

Is the answer correct? Yes. The code grouped by Soil_Type, Fertilizer_Type, and Water_Frequency, computed average Sunlight_Hours per group, and returned the top 5.
Could there be another valid answer? Yes. If the question is interpreted as top 5 individual plant observations rather than condition groups, the ranking would differ. The question is also framed around sunlight as a growth proxy, but another interpretation might rank by Temperature instead.

Is the question Ambiguous? Yes. Highest average Sunlight_Hours is a proxy for growth-conducive conditions, but sunlight is an input variable, not an outcome.
How to rewrite: Group plants by Soil_Type, Fertilizer_Type, and Water_Frequency. For each group, report the average Sunlight_Hours, Temperature, and Humidity. Identify the top 5 groups by average Sunlight_Hours and describe the environmental profile of each.

Plant Advanced Q8 - High Sunlight, Low Humidity

Is the answer correct? Yes. The code correctly applied the top ¼ for sunlight and bottom ¼ for humidity and reported the most common Soil_Type and Temperature.
Could there be another valid answer? Yes. The thresholds for high and low are defined by quantiles. Using fixed absolute thresholds (e.g., Sunlight_Hours > 8 and Humidity < 45) would produce a different subset and potentially different values.
Is the question Ambiguous? Yes. High and low are relative terms without defined thresholds.
How to rewrite: Among plants where Sunlight_Hours is above the 75th percentile and Humidity is below the 25th percentile, report the count, the most frequent Soil_Type, and the mean Temperature.

Plant Advanced Q9- Different Temperature and Humidity

Is the answer correct? Yes. The code grouped by Soil_Type, Water_Frequency, and Fertilizer_Type, computed the range of Temperature and Humidity within each group, and returned the most variable groups.
Could there be another valid answer? Yes. Another approach might normalize the ranges or weight Temperature and Humidity differently.
Is the question Ambiguous? Yes. Similar conditions is subjective, and the question does not specify what threshold constitutes significantly different environmental values.
How to rewrite: Group plants by Soil_Type, Water_Frequency, and Fertilizer_Type. For each group with at least 5 plants, compute the standard deviation of Temperature and Humidity. Report the top 5 groups with the highest combined variability and describe what patterns emerge in those groups.